

RV32IM-Based MCU

Single-Cycle Core with GPIO, Basic Timer, Interrupt Controller
and a Clock-Decoupled Division Accelerator

Course	Advanced CPU Architecture & Hardware Accelerators Lab 361-1-4693
Institution	Ben-Gurion University of the Negev
Submitters	Omer — 208838417 · Ilay — 209128677
Target	Altera DE2-115 — Cyclone IV E EP4CE115F29C7
Toolchain	Quartus Prime Lite 21.1 · ModelSim (VHDL-2008) · RARS
Date	30 August 2026

Contents

1. Introduction and system overview
2. Architecture
 1. Top-level block diagram
 2. Top-level interface
 3. RTL Viewer
 4. Memory map and interrupt vectors
 5. Clock domains
3. HDL source files
4. Design decisions
5. Functional verification (ModelSim)
6. Hardware validation (ISMCE and Signal-Tap)
7. PPA characterization
8. Findings
9. Conclusions

1. Introduction and system overview

This report documents the design, verification and characterization of an RV32IM-based microcontroller implemented on an Altera DE2-115 board. The CPU is a single-cycle RV32IM core in a Harvard organisation, with separate 8 KiB tightly coupled memories for instructions (ITCM) and data (DTCM), both realised in M9K block RAM.

Around the core sits a memory-mapped peripheral set — GPIO driving the LEDs and the six seven-segment displays, a Basic Timer with compare, PWM and input-capture modes, and an interrupt controller with a prioritised vector table. Integer division is offloaded to a dedicated accelerator that runs in a second, faster clock domain, with every crossing between the two domains protected by two-flip-flop synchronisers.

The whole design was verified in three independent ways: against a RARS golden model in simulation, against the same golden model on hardware through the In-System Memory Content Editor, and behaviourally through Signal-Tap traces captured on the board. All three agree.

Property	Value
ISA	RV32IM, single-cycle
CPU clock (MCLK)	25 MHz (PLL c0 from the 50 MHz board oscillator)
Accelerator clock (DIVCLK)	150 MHz (PLL c1) — six times MCLK
ITCM / DTCM	8 KiB each, M9K, runtime-modifiable over JTAG

Peripherals	GPIO, Basic Timer, interrupt controller
Achieved $f_{\max}$	25.84 MHz (MCLK) · 199.6 MHz (DIVCLK)
Area	4,056 logic elements (4 % of the device), 1,905 registers
IPC (RV32IM/test1)	0.798 — identical in simulation and in silicon

Table 1. System summary.

2. Architecture

2.1 Top-level block diagram

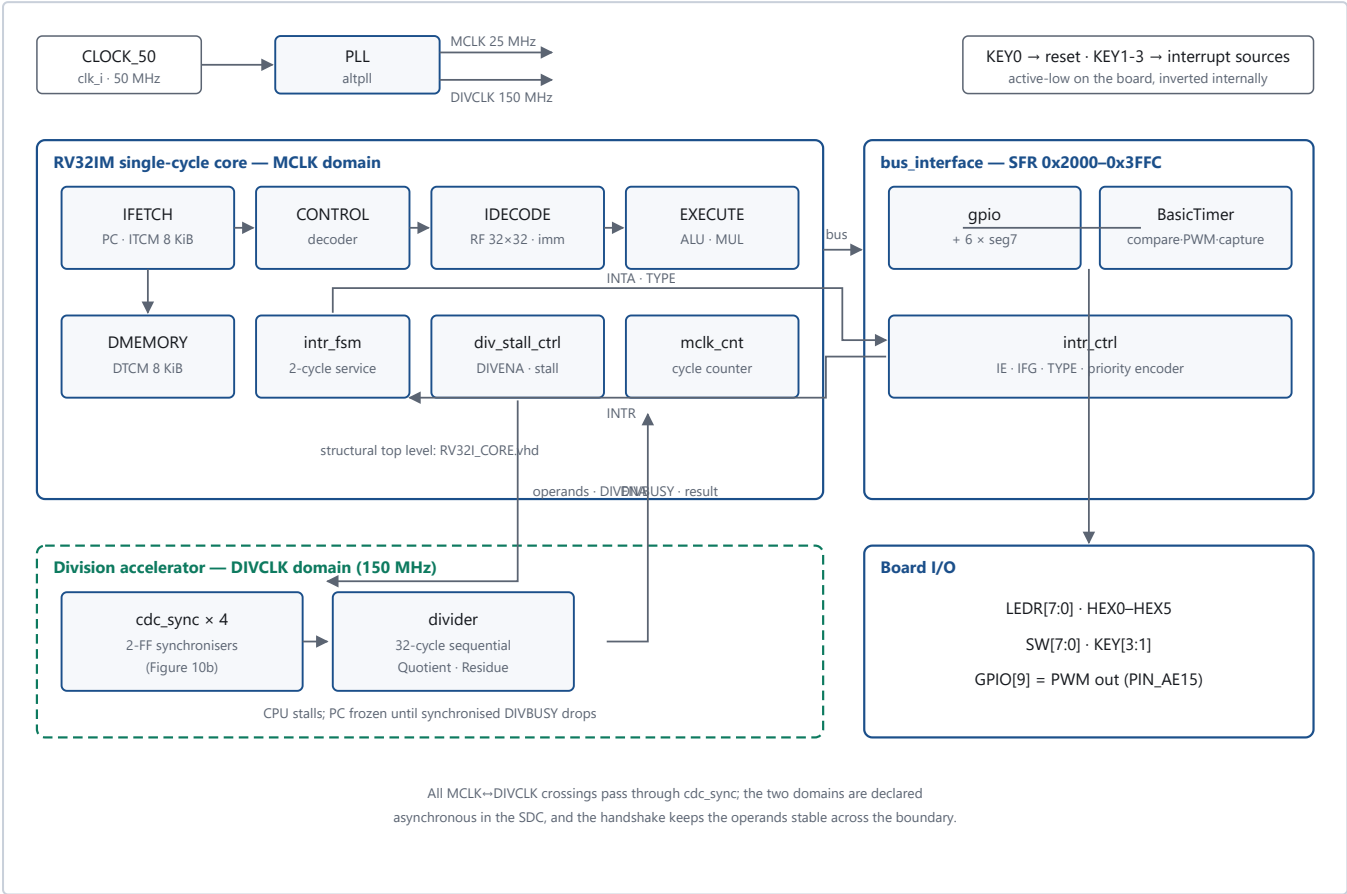


Figure 1. Top-level block diagram. The MCLK domain holds the single-cycle core and the memory-mapped peripherals; the divider runs in the DIVCLK domain and is reached only through two-flip-flop synchronisers.

2.2 Top-level interface

Port	Dir	Width	Description
Clock and reset			
clk_i	in	1	CLOCK_50, the 50 MHz board oscillator; feeds the PLL
rst_i	in	1	KEY0. Active-low on the board and inverted internally, since the design uses an active-high reset
Board I/O (memory-mapped)			
SW_i	in	8	Slide switches, read through the GPIO block
PB_i	in	3	KEY1-KEY3; also the three pushbutton interrupt sources
LEDR_o	out	8	Red LEDs

HEX0_o ... HEX5_o	out	7 each	The six seven-segment displays, driven through seg7
pwm_o	out	1	Basic Timer PWM output, assigned to GPIO[9] (PIN_AE15)
Observation outputs — mapped to virtual pins, used by the testbench and Signal-Tap			
pc_o	out	13	Program counter. Also how the testbench measures stalls: a stall is a cycle in which the PC is held
instruction_o	out	32	Fetches instruction
mclk_cnt_o	out	16	Free-running MCLK cycle counter — the hardware side of the IPC measurement
RegWrite_ctrl_o, MemWrite_ctrl_o, Branch_ctrl_o, brTaken_o	out	1 each	Control strobes from the decoder and the branch unit
read_data1_o, read_data2_o, write_data_o	out	32 each	Register-file read and write ports
alu_res_o	out	32	ALU result
dtdm_addr_o, dtdm_data_wr_o, dtdm_data_rd_o	out	13 / 32 / 32	Data-memory address and data buses
btifg_o, intr_o	out	1 each	Basic Timer interrupt flag and the interrupt request to the CPU

Table 2. Top-level ports of RV32I_CORE. Only the clock, reset and the board I/O consume real device pads — 64 pins. The observation outputs carry VIRTUAL_PIN assignments, which is why the Fitter reports 64 real and 270 virtual pins: they exist so the testbench and Signal-Tap can watch internal state without the design demanding 334 pads on a 256-pad-usable package.

The design is further parameterised by generics defaulted from cond_compilation_package: MODELSIM selects the simulation or FPGA build, WORD_GRANULARITY selects word- or byte-addressing, the four width generics set the memory geometry, and CLK_CNT_WIDTH = 16 sizes the cycle counter.

2.3 RTL Viewer

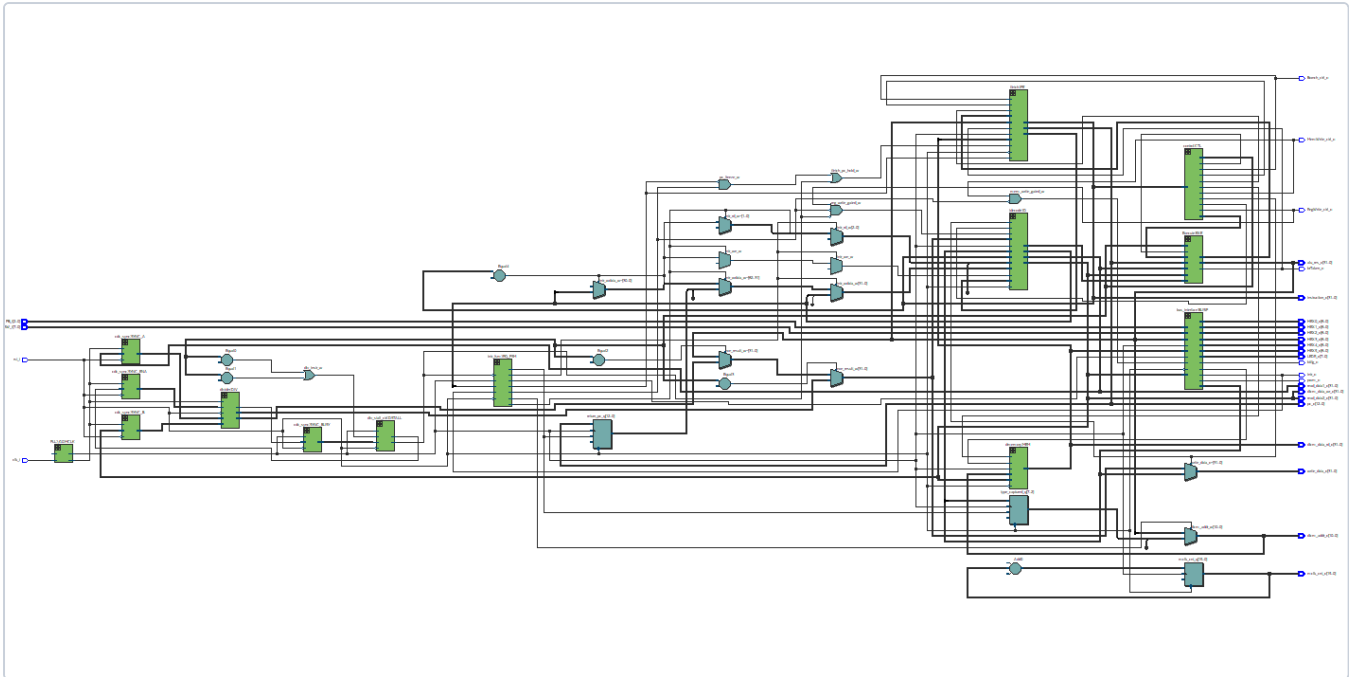


Figure 2. Quartus RTL Viewer of the elaborated top level, confirming the structural instantiation of the core, the bus interface, the interrupt FSM, the accelerator with its synchronisers, and the PLL.

2.4 Memory map and interrupt vectors

The address decode is partial: only bits 15:14:13 are examined to distinguish DTCM from the peripheral window, so the 64-byte SFR block aliases repeatedly across `0x2000-0x3FFF`. This is deliberate — the decode sits on the critical path that gates the DTCM write enable, and a full comparison there would cost frequency.

Region	Address	Contents
DTCM	<code>0x0000-0x1FFC</code>	data memory, 8 KiB; the interrupt vector table occupies the first 32 bytes
GPIO	<code>0x2004-0x200D</code>	LEDR, HEX0-HEX5, SW, pushbuttons
Basic Timer	<code>0x201C-0x2028</code>	BTCTL1, BTCTL2, BTCMPRO/1, BTCAPR
Interrupt controller	<code>0x202C-0x202E</code>	IE, IFG, TYPE

Table 3. Memory map.

Source	Flag	TYPE	Priority	Vector word
Reset	—	00h	—	DTCM[0]
UART error / RX / TX	—	04-0Ch	1-3	DTCM[1..3]
Basic Timer	BTIFG	10h	4	DTCM[4]
Pushbutton 1	KEY1	14h	5	DTCM[5]
Pushbutton 2	KEY2	18h	6	DTCM[6]
Pushbutton 3	KEY3	1Ch	7 (lowest)	DTCM[7]

Table 4. Interrupt vector table. TYPE is emitted already multiplied by four, so it is directly the byte address of the vector entry and the CPU's emulated `lw Mem[TYPE]` needs no shift.

2.5 Clock domains

The PLL produces two clocks from the 50 MHz oscillator: MCLK at 25 MHz for the CPU and peripherals, and DIVCLK at 150 MHz for the divider. Three signals cross from MCLK into DIVCLK (the two 32-bit operands and the DIVENA start pulse) and one crosses back (DIVBUSY); all four pass through `cdc_sync`, the two-flip-flop structure of the project definition's Figure 10b.

The quotient and residue cross back unsynchronised, which is safe by construction:

`div_stall_ctrl` holds the CPU stalled until the *synchronised* DIVBUSY has dropped, so the result buses have long settled before the CPU samples them. This is the standard "synchronise the control, let the settled data cross" handshake, and it is why the SDC declares the two domains asynchronous — timing them against one another would report violations that the synchronisers exist precisely to make irrelevant.

3. HDL source files

File	Description
Packages	
<code>cond_compilation_package.vhd</code>	Conditional-compilation constants: ModelSim vs Quartus selection, memory geometry, and the two PLL ratios (MCLK and DIVCLK).
<code>const_package.vhd</code>	ISA constants — opcodes, funct3/funct7, ALU operation codes. Uses VHDL-2008 sized bit-string literals.
<code>aux_package.vhd</code>	Component declarations for every entity in the design.
CPU core	
<code>RV32I_CORE.vhd</code>	Structural top level. Instantiates the core stages, bus interface, interrupt FSM, accelerator with its synchronisers, and the PLL. Holds the reset-polarity select and the MCLK cycle counter.
<code>IFETCH.VHD</code>	Program counter, 8 KiB ITCM (altsyncram), and next-PC select for sequential, branch, jal, jalr and interrupt-vector fetch. ITCM is runtime-modifiable (ISMCE).
<code>IDECODE.VHD</code>	32×32 register file, immediate generation for all formats, and the register-file write override used by the interrupt service protocol (tp, and GIE in gp[0]).
<code>CONTROL.VHD</code>	Main decoder: ALU operation, branch, jump, memory and register-write control.
<code>EXECUTE.VHD</code>	ALU, branch condition evaluation, and the RV32M multiplier (four embedded 9-bit multipliers).
<code>DMEMORY.VHD</code>	8 KiB DTCM (altsyncram), clocked on the falling edge of MCLK per Figure 3. Runtime-modifiable (ISMCE).
Memory-mapped I/O	
<code>bus_interface.vhd</code>	SFR address decode, peripheral chip selects, write-strobe distribution, and the read-data multiplexer back to the CPU.
<code>gpio.vhd</code>	LEDR, the six HEX registers, and the SW / pushbutton inputs at their SFR addresses.
<code>seg7.vhd</code>	Binary-to-seven-segment decoder, instantiated six times through a generate statement.
Basic Timer	
<code>BasicTimer.vhd</code>	Structural top of the timer; exposes the control-register bit fields as named aliases and registers the interrupt event so no combinational glitch can clock a phantom BTIFG.
<code>bt_prescaler.vhd</code>	BTSEL divider, implemented as a clock enable rather than a clock multiplexer.
<code>bt_counter.vhd</code>	BTCNT up-counter and the two compares against BTCL0 / BTCL1 producing EUQ0 and EUQ1.
<code>bt_output_unit.vhd</code>	PWM output: set on EUQ1, reset on EUQ0 (Figure 8, Mode 0).

<code>bt_capture.vhd</code>	Input capture: CAPSEL source multiplexer, two-flop synchroniser, edge detection per CAPMD, and the latch of BTCNT into BTCAPR.
<code>bt_regs.vhd</code>	Timer register file: BTCTL1/2, BTCMPR0/1 with their latched shadows BTCL0/BTCL1, and BTCAPR.
Interrupt subsystem	
<code>intr_ctrl.vhd</code>	IE, IFG and TYPE registers, the pending term IFG^IE, the priority encoder, and the INTR output.
<code>intr_fsm.vhd</code>	The CPU-side two-cycle service protocol of page 15: clear GIE, acknowledge, capture TYPE, clear the source flag, and emulate the vector load and jalr with tp holding the return address.
Accelerator	
<code>divider.vhd</code>	32-cycle sequential divider producing Quotient and Residue, running in DIVCLK.
<code>div_stall_ctrl.vhd</code>	Detects div/rem, raises DIVENA, and freezes the PC until the synchronised DIVBUSY drops.
<code>cdc_sync.vhd</code>	The two-flip-flop CDC synchroniser of Figure 10b, instantiated four times. Carries an <code>AUTO_SHIFT_REGISTER_RECOGNITION OFF</code> attribute — see §8.
Clocking	
<code>PLL.vhd</code>	altpll wrapper: c0 = MCLK (25 MHz), c1 = DIVCLK (150 MHz).

Table 5. The 24 design source files.

4. Design decisions

4.1 BTSSEL as a clock enable, not a clock multiplexer

Figure 7 draws BTSSEL as a 4:1 multiplexer selecting between clock sources. Implemented literally on an FPGA this produces a derived clock off the global network, which fragments the clock tree and makes $f_{\max}$ unreportable. `bt_prescaler` instead generates a single-cycle *tick* enable at the selected rate, and BTCNT advances on that enable. The behaviour is identical; the implementation stays synthesis-clean and the whole timer remains in one analysable clock domain.

4.2 The divider in its own clock domain

A 32-cycle divider clocked at MCLK would stall the CPU for 32 cycles per operation. Running it at 150 MHz — six times MCLK — reduces that to **8 MCLK cycles**, measured. On RV32IM/test1, which performs sixteen divide-class operations, this is the difference between IPC 0.50 and IPC 0.80 with an identical retired-instruction count. DIVCLK was set to 150 MHz rather than higher to leave margin under the divider's standalone ceiling of ~195 MHz.

4.3 Registering the timer interrupt event

EUQ0 is combinational over signals that settle in different delta cycles, so it emitted glitches narrower than a clock period. Figure 13 uses that line to *clock* the IFG flip-flop, where such a glitch latches a phantom interrupt. The event is therefore registered inside `BasicTimer` before it leaves the block. This was found in simulation, where a timer interval measured 7 cycles instead of 80.

4.4 Stack pointer initialisation

The benchmarks ship without a C runtime startup, so `sp` reset to zero. A compiled program's locals then landed near `0xFFFFFxx`, whose bits 15:14:13 are all '1' and are therefore decoded as *peripheral*: stack writes were discarded and stack reads returned an I/O port, so loop counters never advanced. `x2` is now reset to `0x2000`, the top of the DTCM. Hand-written assembly benchmarks never touch the stack, which is why this stayed hidden until the first compiled benchmark ran.

4.5 One testbench, many run scripts

The project definition names a single `tb_RV32IMscMCU.vhd`. Everything that differs between benchmarks — memory images, run length, board stimulus — therefore lives in the per-benchmark `.do` files, and the testbench leaves SW and the pushbuttons undriven for the script to force.

4.6 Producing the golden-model artefact

The DTCM is an altsyncram whose storage ModelSim ASE's `mem save` cannot reach. The testbench therefore mirrors the memory: it pre-loads a shadow array from the same image the hardware memory is initialised from, updates it on every committed store, and writes it out as `DTCM.mem` in the one-word-per-line format of the RARS golden file.

4.7 Measuring IPC without touching the architecture

A stall is, by definition, a cycle in which the PC is held. Since `pc_o` is already a top-level output, both IPC counters are accumulated in the *testbench* rather than in the design — no hardware counter was added and the architecture is unchanged. On hardware the existing `mclk_cnt_o` output serves the same purpose.

5. Functional verification (ModelSim)

5.1 Golden-model comparison

Of the eight supplied benchmarks only RV32IM/test1 terminates, and it is correspondingly the only one shipping a RARS post-run reference — the interactive benchmarks sit in a state-machine loop forever, so there is no end-of-run state to dump. `run_rv32im_test1.do` performs the comparison automatically and reports **PASS on all 1024 DTCM words**. The remaining seven are verified behaviourally from their waveforms.

5.2 IPC

Following LAB5 clause 6.iii.b, referenced by steps 8.c.iii and 8.e.iii:

$$\begin{aligned} \text{IPC} &= (\text{CLKCNT_o} - (\text{STCNT_o} + 4 + \text{depth} \cdot \text{FHCNT_o})) / \text{CLKCNT_o} \\ &= \text{InstructionCounter} / \text{CLKCNT_o} \end{aligned}$$

A single-cycle core has no flushes and no pipeline fill, so this reduces to $\text{IPC} = (\text{CLKCNT} - \text{STCNT}) / \text{CLKCNT}$.

Benchmark	CLKCNT	STCNT	Retired	IPC
RV32IM/test1	643	130	513	0.798
GPIO/test0	19,992	2	19,990	0.9999
Interrupt/test1	7,852	2	7,850	0.9997
Interrupt/test4	7,852	87	7,765	0.9889

Table 6. IPC across benchmarks. RARS reports **513** instructions executed for RV32IM/test1 and the hardware retired **513**, so both sides of the equation agree exactly and the check passes.

STCNT for test1 decomposes as sixteen stalling operations (eight `div`, eight `rem`) at 8 cycles each, plus two cycles of edge effects. With the divider clocked at MCLK the same program would need 32 cycles per operation, giving CLKCNT 1,027 and IPC 0.500. Because the retired-instruction count is identical in both cases, the improvement from 0.50 to 0.80 is an exact comparison rather than an estimate.

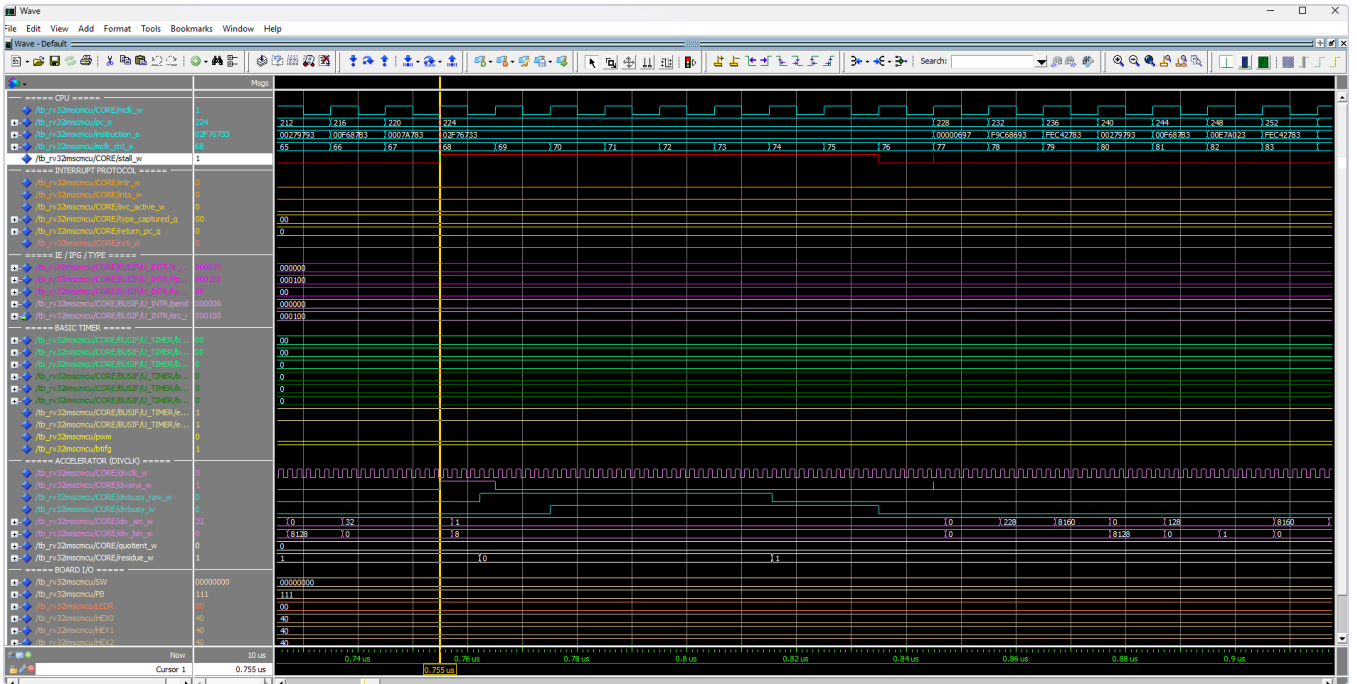


Figure 3. RV32IM/test1 — the divider accelerator servicing `rem a4,a4,a5 (0x02F76733)`. The CPU stalls for 8 MCLK cycles with the PC held while the divider runs 32 cycles in the 150 MHz domain. `divbusy_raw_w` (DIVCLK) leads `divbusy_w` (MCLK) by the two-flop synchroniser of Figure 10b, so this single frame evidences Figures 9, 10a and 10b together.

5.3 GPIO benchmarks

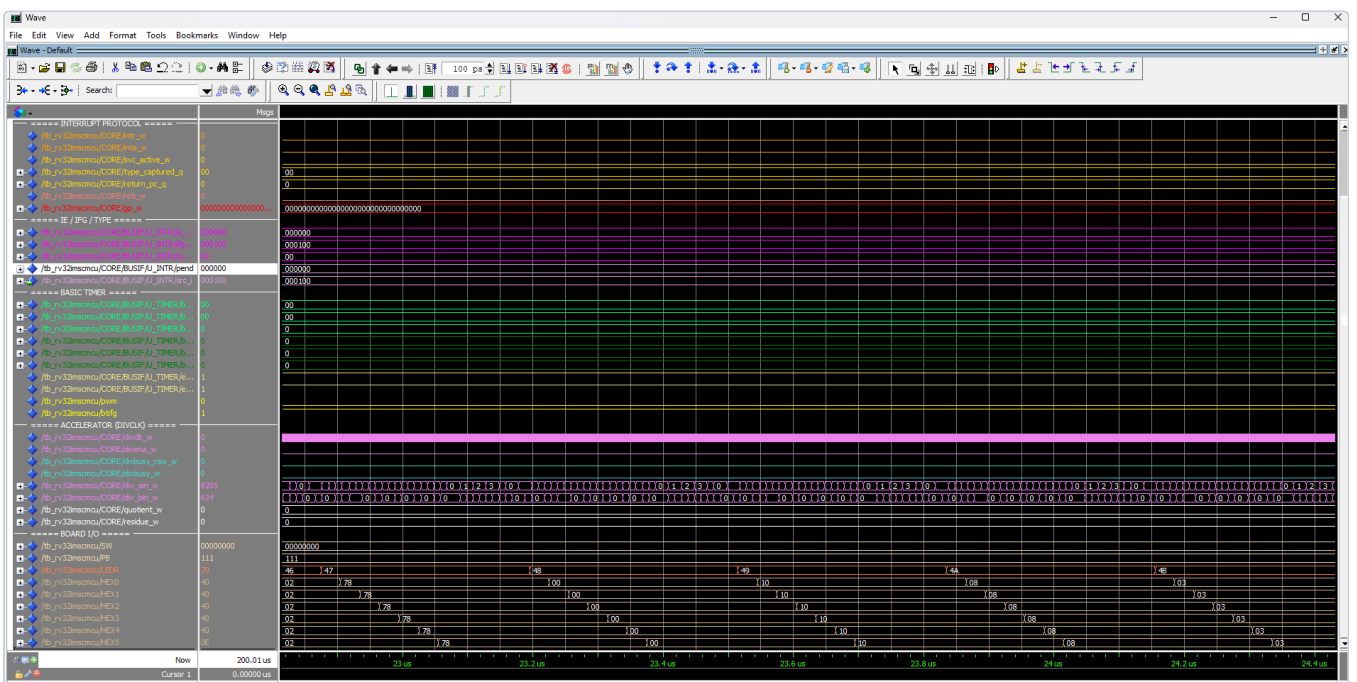


Figure 4. GPIO/test0 — LEDR counts 0x46→0x4B and each HEX digit is written in turn. The transitions are *staggered*, one per store to 0x2004/2005/2008/2009/200C/200D, so the memory-mapped write path is visible in program order. IE = 000000 throughout: the free-running BTIFG is set but masked, which incidentally demonstrates that the IE mask works and that no interrupt perturbed the measurement.

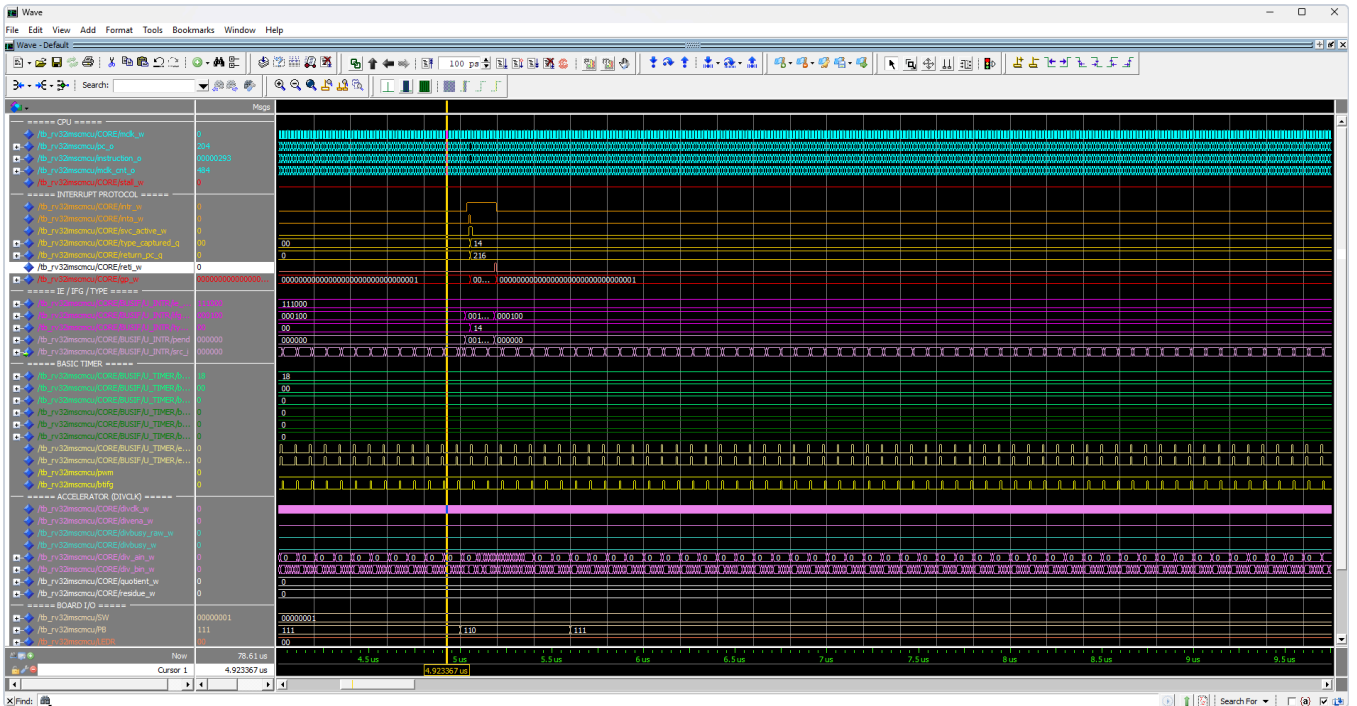


Figure 7. Interrupt/test1 — a KEY1 interrupt serviced end to end. IFG bit 3 sets on the press, the pending term passes the IE mask, INTR is raised and acknowledged. During the two-cycle service the CPU clears GIE in hardware and latches TYPE = 14h with return address 216; `reti` restores GIE and the PC resumes. The GIE bit going **1** → **0** → **1** demonstrates spec notes (e) and (f) rather than asserting them.

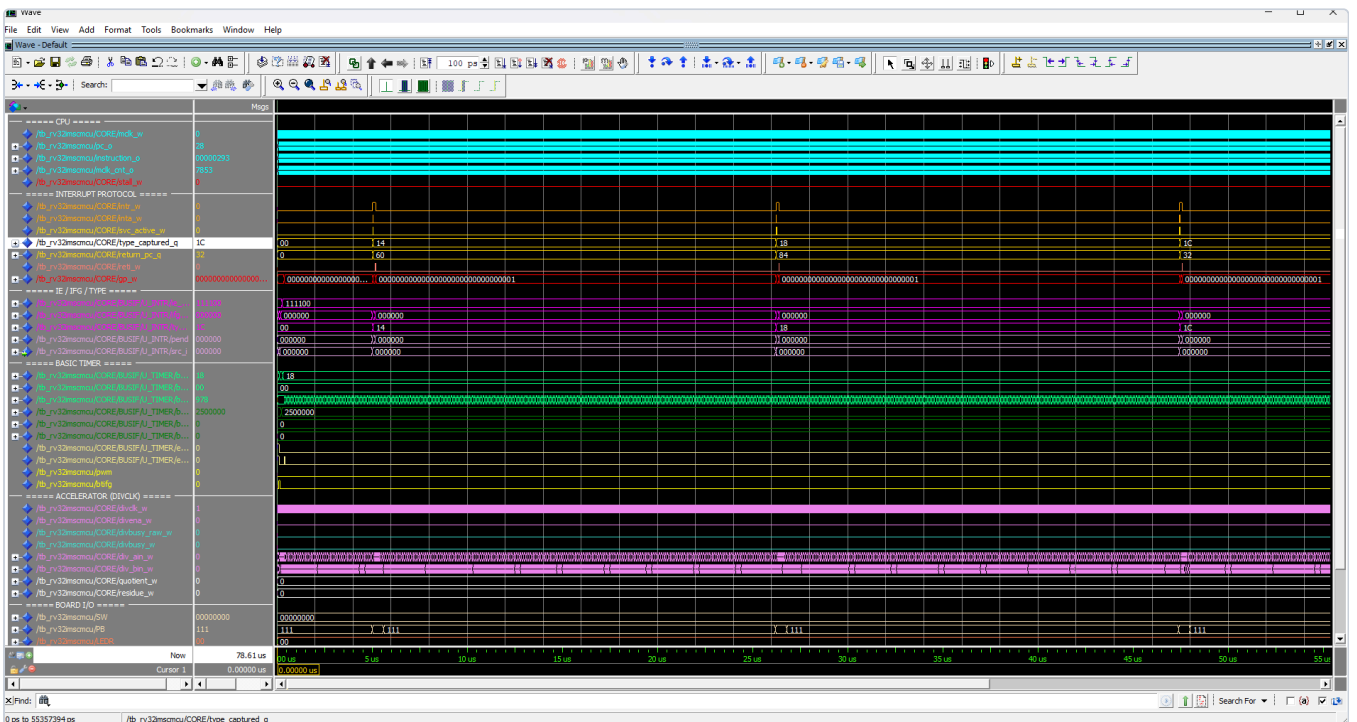


Figure 8. Interrupt/test2 — vector dispatch. Three presses drive TYPE through 14h / 18h / 1Ch with the return address changing each time, proving the priority encoder and the vector table. The timer is configured BTCTL1 = 0x18 (SMCLK/8) with BTCMPRO = 2,500,000.

6. Hardware validation (ISMCE and Signal-Tap)

Both TCMs carry `ENABLE_RUNTIME_MOD`, so all eight benchmarks were run from a *single* compilation: the In-System Memory Content Editor writes each benchmark's ITCM and DTCM images over JTAG, and KEY0 restarts the program. This removes the per-benchmark recompile entirely.

6.1 RV32IM/test1 — golden model and IPC in silicon

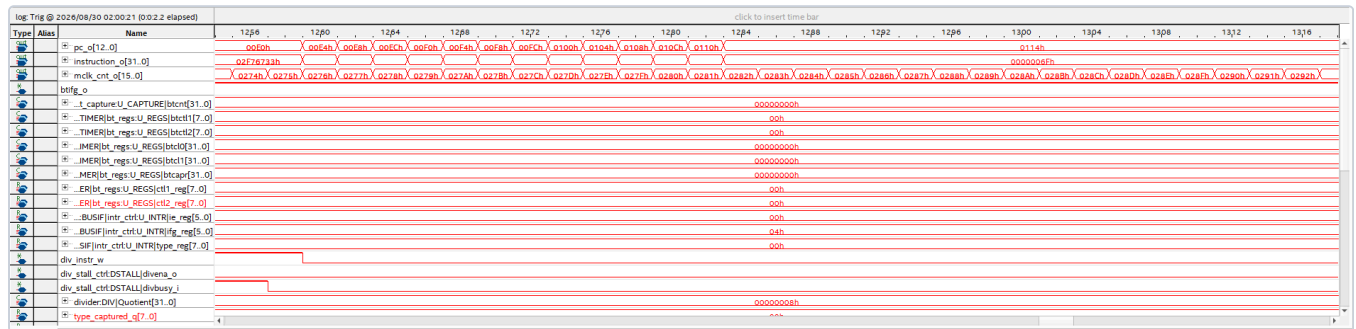


Figure 11. RV32IM/test1 captured on the board. The program reaches its terminal loop at `pc_o = 0114h` holding `0000006Fh` (`jal x0,0`), and `mclk_cnt_o` reads `0283h = 643` — identical to the simulated cycle count. The same frame shows the accelerator: `02F76733h` (`rem a4,a4,a5`) held at `pc_o = 00E0h` while the stall is asserted, then the PC advancing one address per cycle the moment it drops. Note also `ie_reg = 00h` with `ifg_reg = 04h`: the free-running BTIFG is set but correctly masked, so no interrupt perturbed the cycle count.

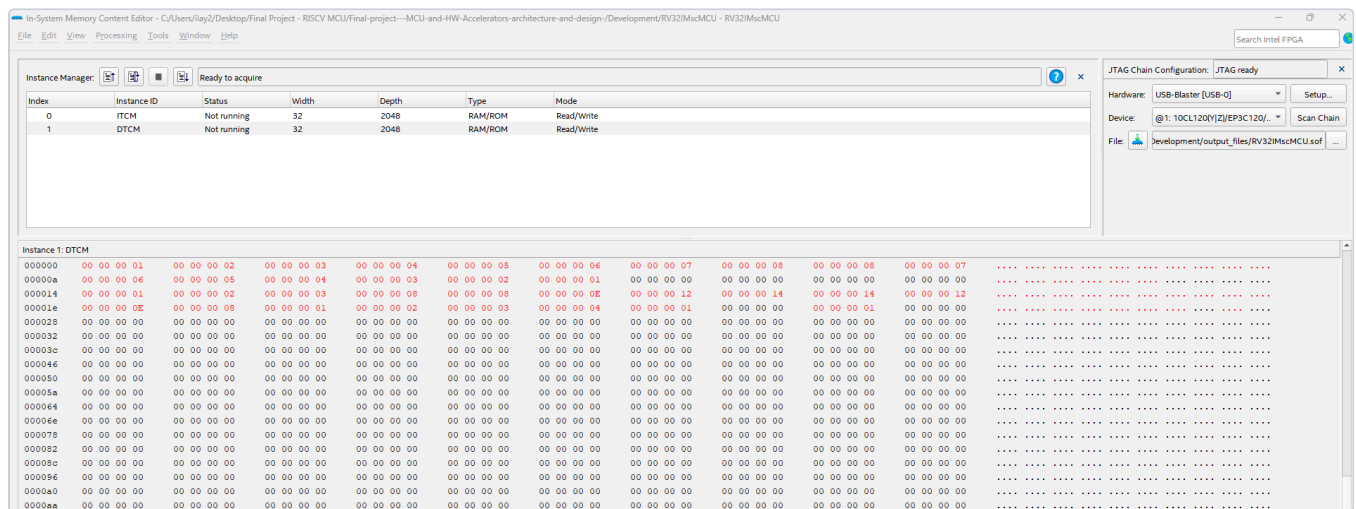
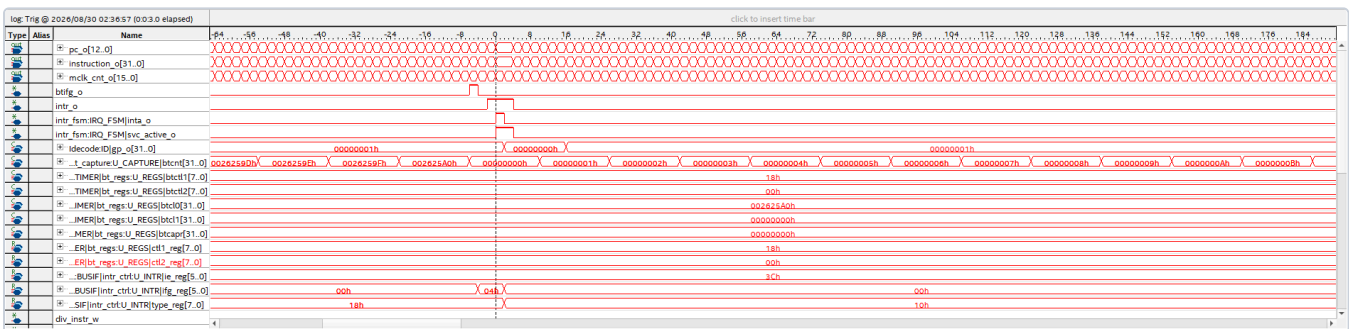
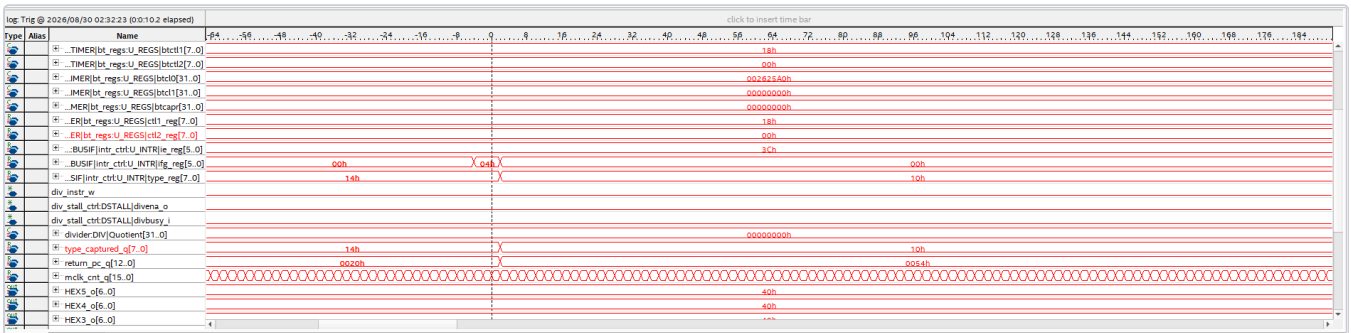
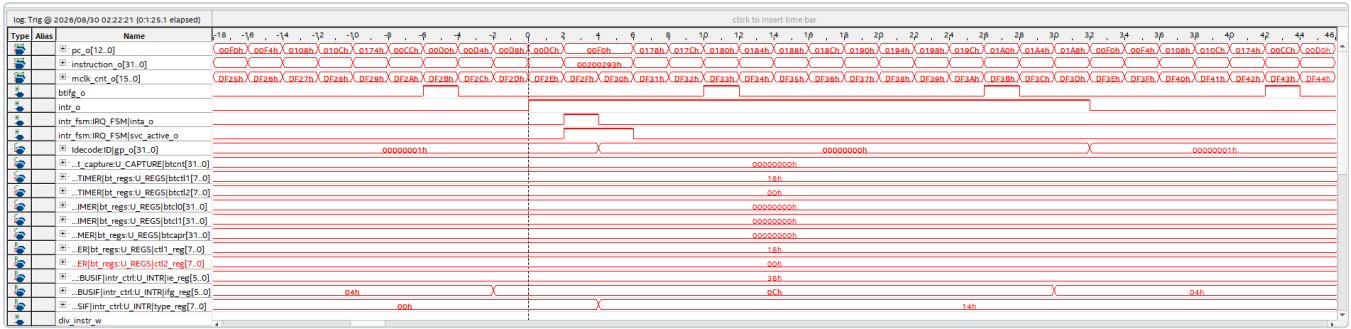


Figure 12. ISMCE readback of the DTCM after the run, compared word by word against the RARS `DTCM_golden.txt`: **exact match**. Words 0–7 hold the first array 1..8, words 8–15 the second 8..1, then the computed results. This satisfies step 8(e).i.

Simulation and silicon agree exactly. Hardware 643 cycles = simulation 643 cycles = RARS 513 retired instructions, giving IPC 0.798 by both sides of the equation, and the DTCM contents match the golden model bit for bit.

6.2 Interrupt benchmarks on hardware



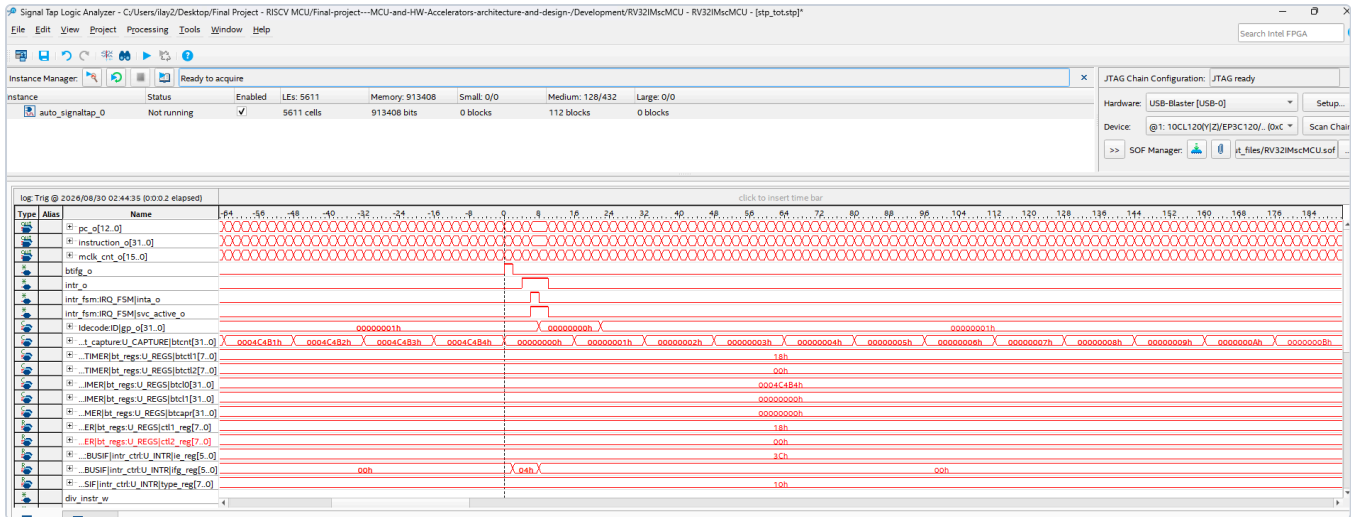


Figure 17. Interrupt/test3 — BTCL0 = 0004C4B4h = 312,500, the shortest programmed interval, with BTCNT reaching the compare value and wrapping. The longest interval, 002625A0h = 2,500,000, was captured the same way. Simulation (Figure 9) shows all four intervals; hardware confirms the extremes and the end-to-end behaviour.

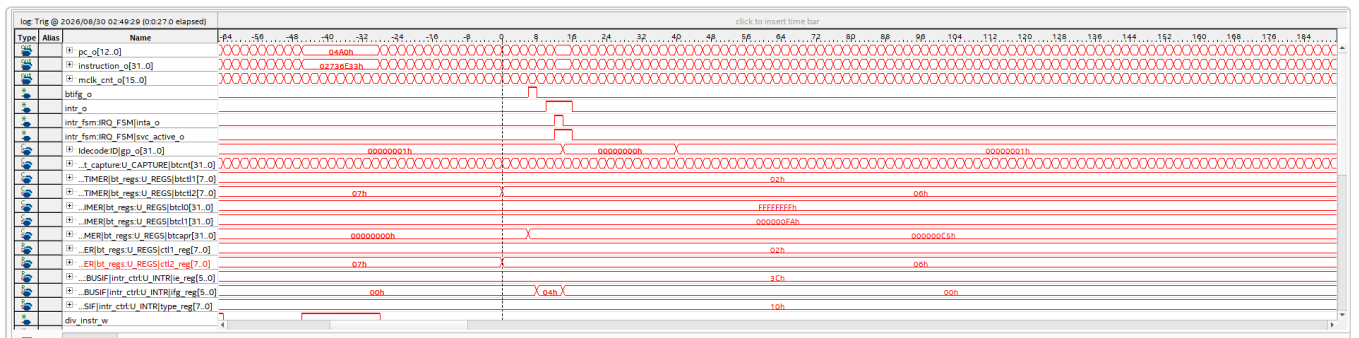
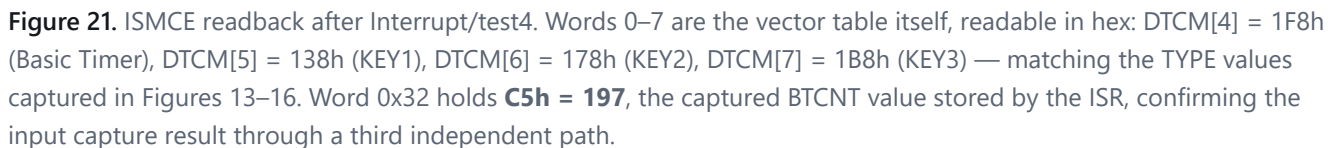
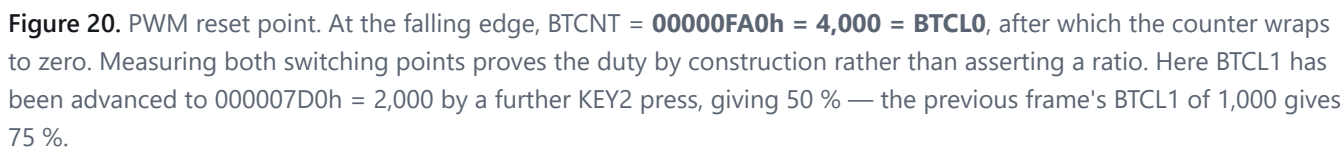


Figure 18. Interrupt/test4, input capture — four subsystems in one frame. BTCTL2 goes 07h → 06h (CAPISEL switching from GND to VCC, the software-generated edge), **BTCAPR latches C5h = 197** — identical to the simulated value — BTCL0 is FFFFFFFFh (free-running), and in the same window the divider stalls the CPU on 02736E33h (rem t3,t1,t2) at pc_o = 04A0h.



Figure 19. PWM set point. At the rising edge of the output, BTCNT = 00003E8h = 1,000 = BTCL1 — the output sets at BTCL1, exactly as Figure 8 Mode 0 specifies. (The one-cycle offset is the output register inside bt_output_unit.)



Quantity	Simulation	Hardware	Agreement
RV32IM/test1 cycle count	643	643	exact
RV32IM/test1 DTCM contents	1024/1024 vs RARS	matches RARS golden	exact
Interrupt vector set (TYPE)	10h/14h/18h/1Ch	10h/14h/18h/1Ch	exact
Service latency	2 cycles	2 cycles	exact
Input capture BTCAPR	197	197	exact
PWM switching points	BTCL1 / BTCL0	1,000 / 4,000	exact

Table 7. Simulation versus silicon.

7. PPA characterization

All figures are post-Fitter for the final build with Signal-Tap disabled, slow 1200 mV 85 °C timing model, device EP4CE115F29C7. The *MCU with GPIO* row is derived from the Fitter's per-entity resource report of the full design (Figure 23) by removing the timer, interrupt subsystem, accelerator and its synchronisers; the GPIO-only variant was not built as a separate project. The pipelined variant was not attempted.

7.1 Area

	Logic elements	Registers	I/O pins	Memory bits	9-bit multipliers	PLLs
MCU with GPIO	3,402	1,337	63	131,072	4	1
MCU with GPIO and Interrupt Capability	4,056	1,905	64	131,072	4	1
Pipelined MCU with GPIO and Interrupt Capability	not attempted (bonus)					

Table 8. Area. The full design occupies 4 % of the device.

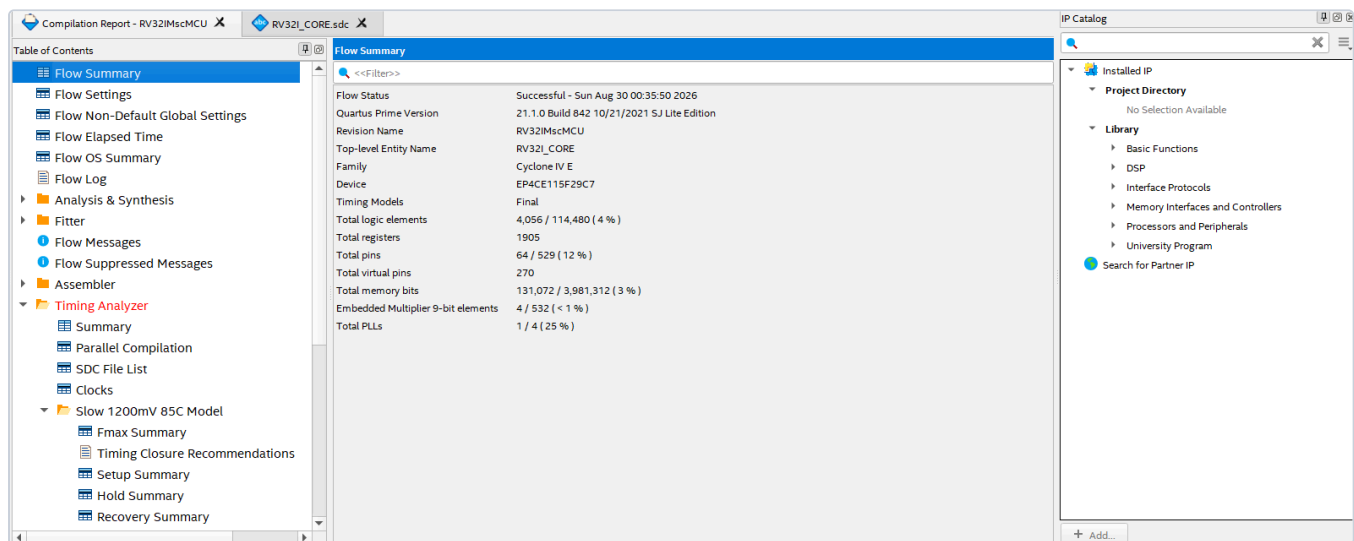


Figure 22. Quartus area report for the final build.

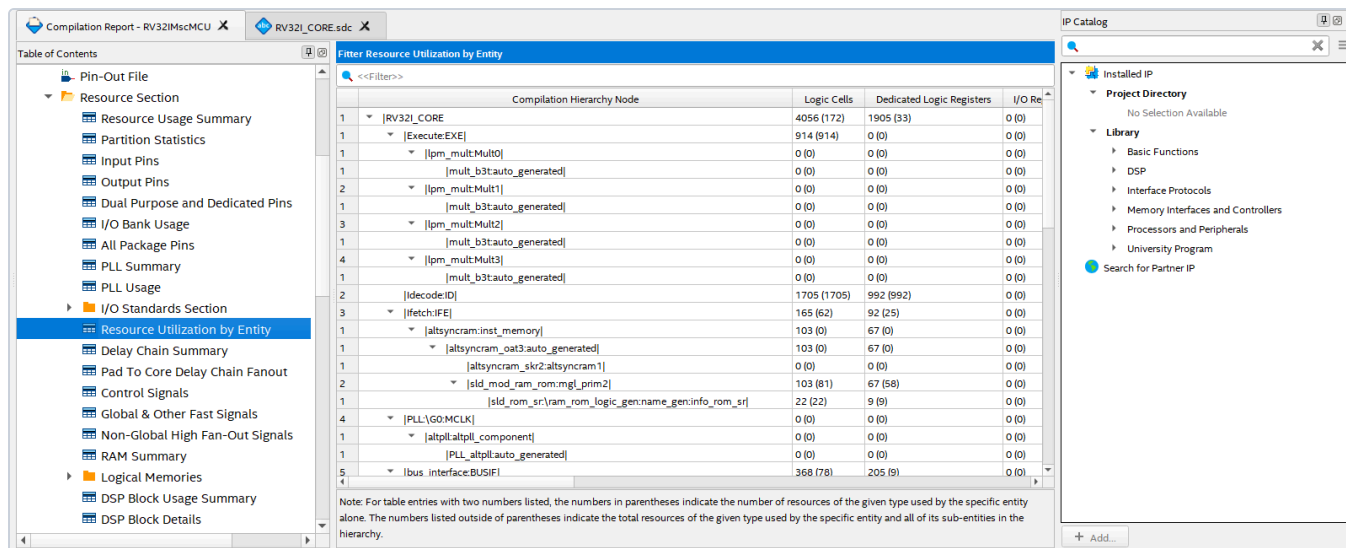


Figure 23. Fitter resource utilization by entity — the source of the per-block costs in Table 9.

Block added beyond a GPIO-only MCU	Logic elements	Registers
BasicTimer	202	148
divider	281	267
cdc_sync × 4	132	132
intr_ctrl	31	16
intr_fsm	4	3
div_stall_ctrl	4	2
Total	654	568

Table 9. Incremental cost of the timer, interrupt subsystem and accelerator: 654 logic elements, about 16 % on top of a 3,402-element GPIO baseline. For scale, IDECODE alone costs 1,705 elements — the 32×32 register file dominates the design.

7.2 Performance

	$f_{\max}$	Critical path — slowest submodule and why	f_{MCLK}
MCU with GPIO	25.84 MHz	Identical to the row below: the limiting path lies inside the core, not in the peripherals, so removing them does not raise $f_{\max}$.	25 MHz
MCU with GPIO and Interrupt Capability	25.84 MHz (MCLK) 199.6 MHz (DIVCLK)	ITCM address register → DTCM write-enable register. The DTCM is clocked on the falling edge of MCLK per Figure 3, so this is a cross-edge path that receives only <i>half</i> a clock period — 20.000 ns of a 40 ns clock against 19.997 ns of delay. It spans instruction fetch, decode, ALU address generation and the memory-mapped address decode, plus two M9K accesses. The combinational logic itself would support roughly 50 MHz on a full period; the halving is the entire reason the MCU sits at 25 MHz.	25 MHz
Pipelined MCU with GPIO and Interrupt Capability	not attempted (bonus)		

Table 10. Performance. Worst-case setup slack is +0.648 ns on MCLK and +1.656 ns on DIVCLK, both with zero total negative slack.

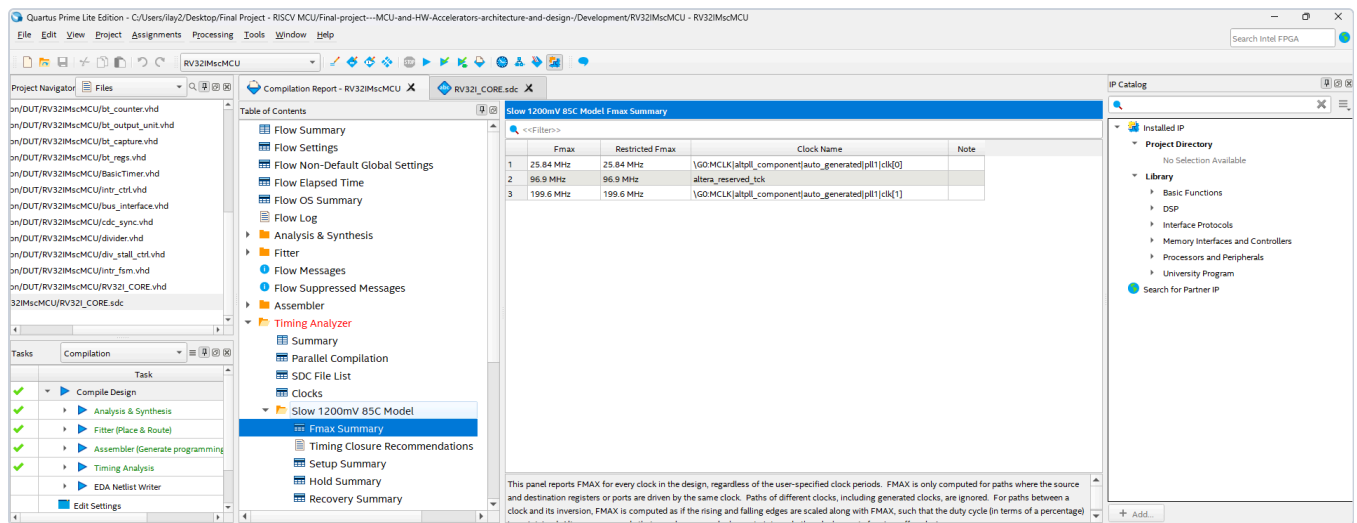


Figure 24. Quartus $f_{\max}$ report — both PLL output clocks.

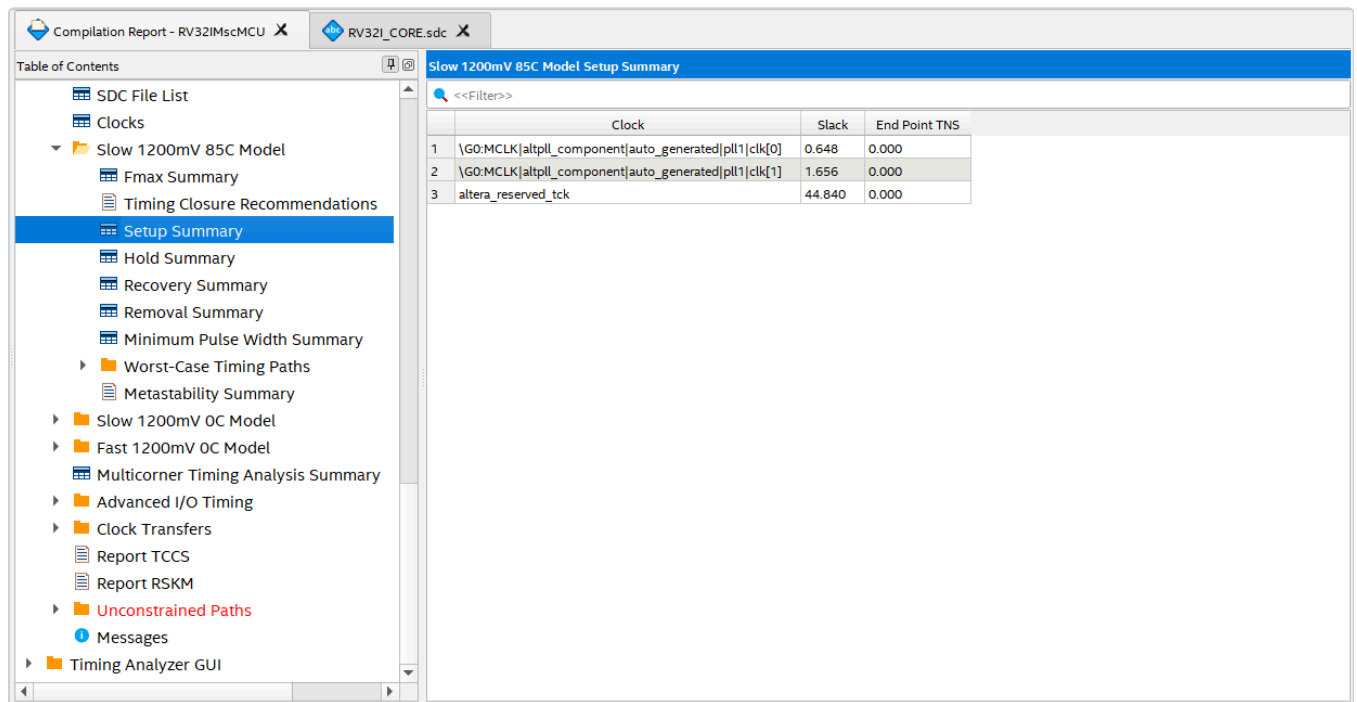


Figure 25. Setup summary: MCLK +0.648 ns, DIVCLK +1.656 ns, TNS zero on both.

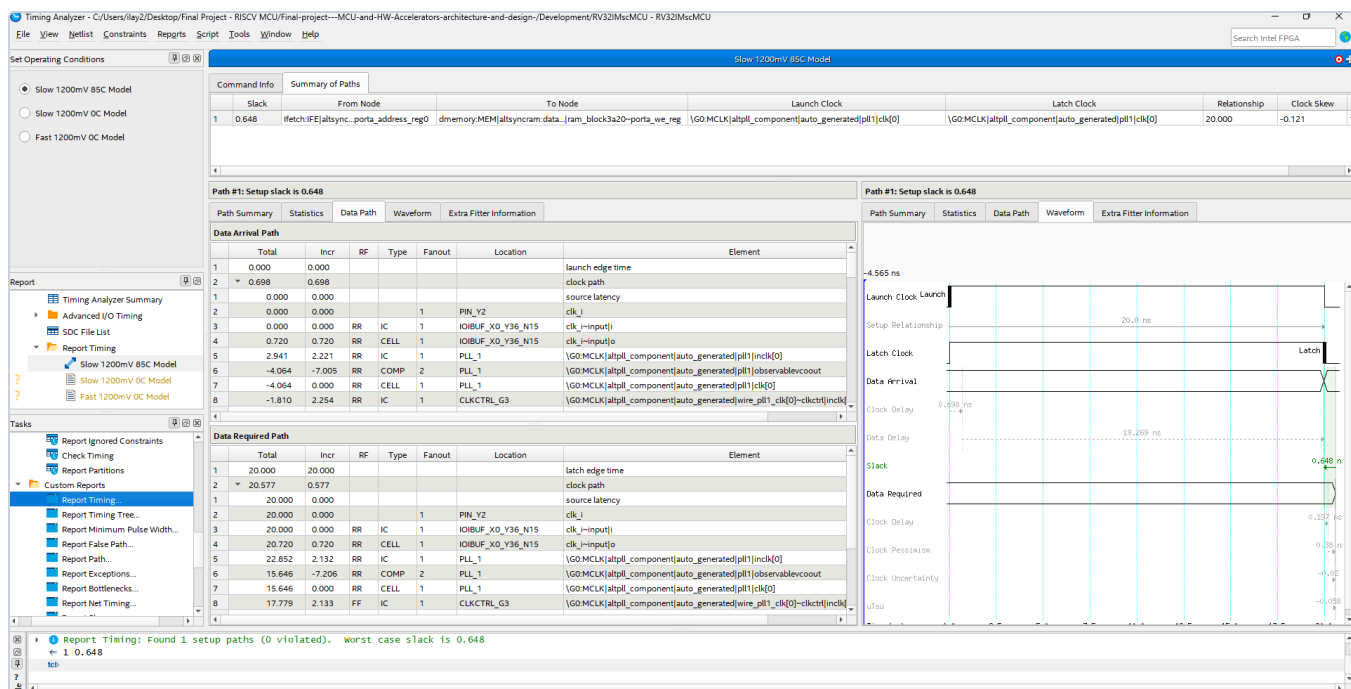


Figure 26. The worst-case setup path, showing the 20.000 ns launch-to-latch relationship that reveals it as a half-period cross-edge path.

7.3 Power

	Total	Static	Dynamic	I/O
MCU with GPIO	192.30 mW	102.19 mW	42.07 mW	48.04 mW
MCU with GPIO and Interrupt Capability	201.15 mW	102.19 mW	50.16 mW	48.80 mW
Pipelined MCU with GPIO and Interrupt Capability	not attempted (bonus)			

Table 11. Thermal power dissipation. The first row follows the same derivation as Table 8: static power is device leakage and essentially constant, I/O power scales with pin count, and core dynamic power scales with the logic-cell ratio. Static power dominates at just over half the total, which is expected for a design occupying 4 % of a large device at 25 MHz.

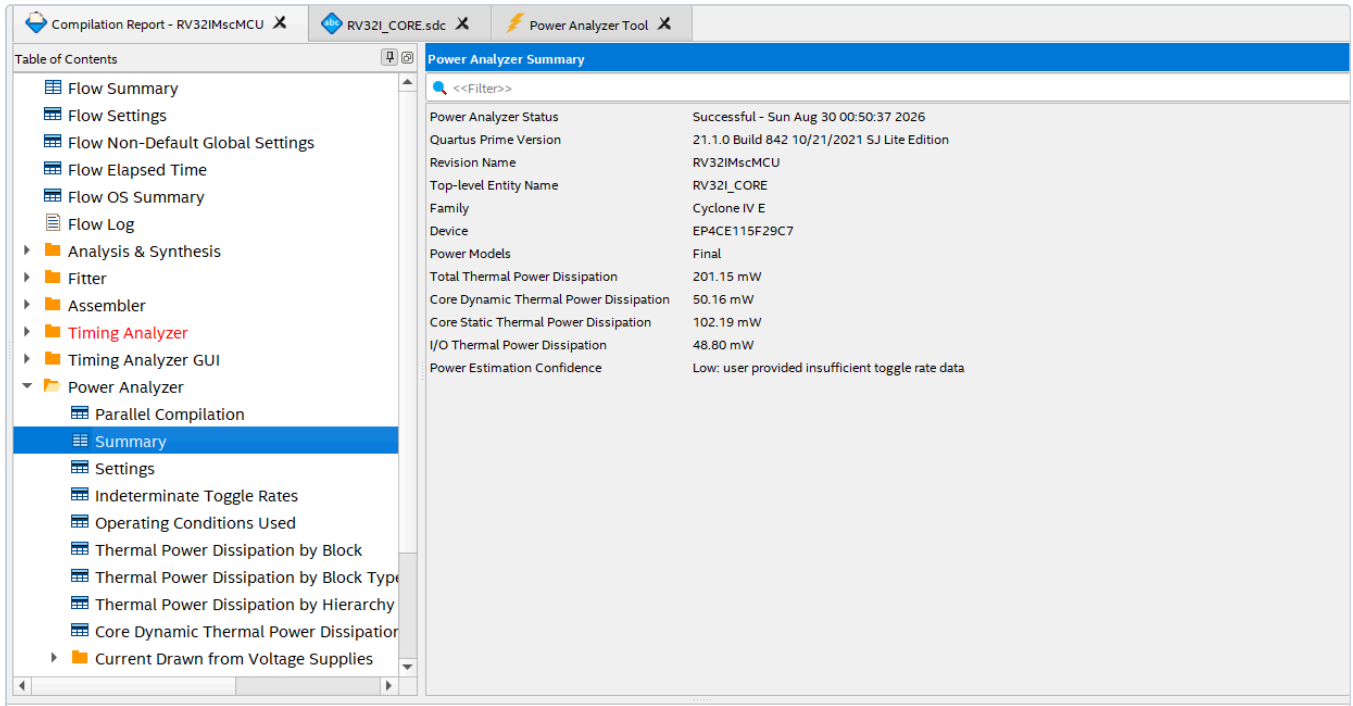


Figure 27. Quartus Power Analyzer report. The analyser reports low confidence because no switching-activity file was supplied, so the figures use default toggle rates.

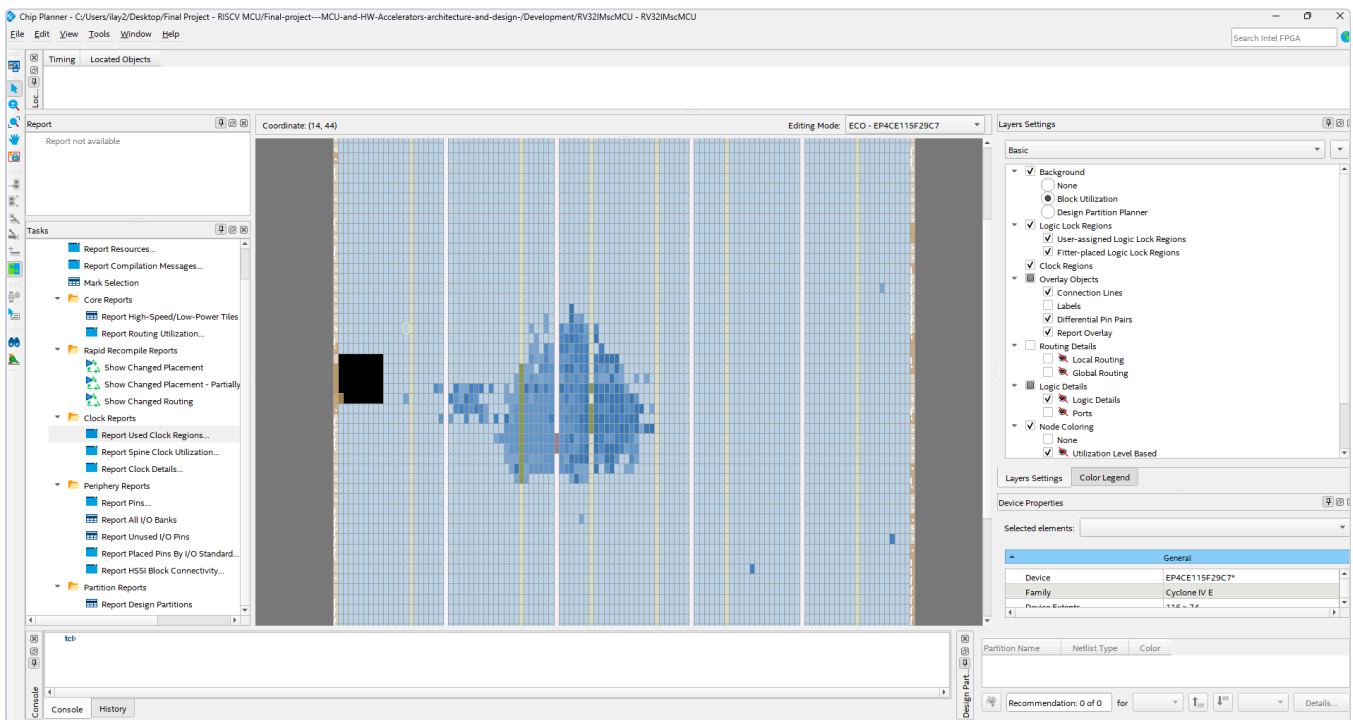


Figure 28. Chip Planner floorplan of the placed design.

8. Findings

8.1 The Figure 10b synchronisers were being implemented in RAM

Inspecting the Fitter's per-entity report revealed that `cdc_sync`'s two stages had been recognised as a shift register and mapped into *two M9K blocks* — Quartus had merged the three MCLK→DIVCLK instances ($32 + 32 + 1 = 65$ bits, 2 deep) into a single 2×65 ALTSYNCRAM.

This is wrong for a synchroniser. The first stage exists so that a metastable sample lands in a dedicated flip-flop and is given a full destination-clock period to resolve; a memory cell read back through an address counter does not provide that guarantee. Figure 10b draws two flip-flops, so two flip-flops is what must be built. Adding an `AUTO_SHIFT_REGISTER_RECOGNITION OFF` attribute to the architecture fixed it — and improved *both* clock domains, because the RAM lookup had been sitting on both critical paths:

	Before	After
MCLK $f_{\max}$ / slack	25.00 MHz / +0.003 ns	25.84 MHz / +0.648 ns
DIVCLK $f_{\max}$ / slack	185.32 MHz / +1.270 ns	199.6 MHz / +1.656 ns
Memory bits	131,202	131,072 (exactly the two TCMs)
Registers	1,773	1,905

Table 12. Effect of disabling shift-register inference on the CDC synchronisers. Two M9K blocks were recovered, 132 real flip-flops appeared, and both domains got faster.

8.2 Partial address decode

`is_peripheral` examines only address bits 15:14:13, so the 64-byte SFR window repeats 128 times across `0x2000–0x3FFF`. This is a standard technique and deliberate here for timing, since the decode gates the DTCM write enable on the critical path — but it removes fault containment and was the direct cause of the stack-pointer bug in §4.4. A cleaner design would use the full comparison to qualify the peripheral *chip selects* only, never `dtcm_we`.

8.3 Benchmark constants assume a 20 MHz clock

The interrupt benchmarks hardcode timing constants for a 20 MHz MCLK. At the delivered 25 MHz, BTCTL1 = 18h selects $SMCLK/8 = 3.125$ MHz, so BTCMPRO = 2,500,000 produces **0.8 s** rather than the 1.000 s the source comments claim; test3's four intervals become 0.8 / 0.4 / 0.2 / 0.1 s, and test4's nominal 5 kHz PWM measures 6.25 kHz. This is a property of the benchmark constants, not of the design.

8.4 PWM duty convention

The Interrupt/test4 source comments describe the duty cycle as 2^{-k} , which suggested a polarity conflict with Figure 8 Mode 0. The benchmark's own ReadMe resolves it: the intended sequence is 50 %, 75 %, 87.5 %, 93.75 % — that is $1 - 2^{-k}$, which is exactly $(BTCL0 - BTCL1) / BTCL0$, the Mode 0 formula. Figures 19 and 20 confirm it in silicon by measuring both switching points. The source comment is simply inconsistent with the ReadMe.

8.5 INTA naming

`intr_fsm` drives `inta_o` as an internal active-high acknowledge strobe, asserted during the first service cycle, rather than as the bus-level INTA drawn on page 15. The service *sequence* is implemented exactly as specified — two cycles, GIE cleared, TYPE captured from the data bus, source flag cleared, vector loaded and jumped to with tp holding the return address — but a reader comparing signal levels against the figure should be aware of the polarity convention.

8.6 Benchmark issues confirmed independently

Several defects identified while bringing the benchmarks up were subsequently fixed in the supervisor's revision of 29 August 2026: CAPSEL never switching to VCC, an inverted loop guard, a missing jump, BTHOLD left asserted, and a period constant that was eight times too large for SMCLK/8. Figure 18 shows the corrected capture behaviour working.

9. Conclusions

The design meets the specification. An RV32IM single-cycle core with GPIO, a Basic Timer supporting all three modes, a prioritised interrupt controller implementing the two-cycle service protocol, and a clock-decoupled division accelerator all fit in 4 % of the device and close timing at 25 MHz with 0.648 ns of margin.

The accelerator is the main architectural result. Placing the divider in a 150 MHz domain rather than clocking it at MCLK cuts the stall from 32 cycles to 8 and lifts IPC on RV32IM/test1 from 0.500 to 0.798. Because the retired-instruction count is unchanged, this is an exact comparison. The cost is 281 logic elements for the divider and 132 more for the synchronisers that make the crossing safe — around 10 % of the design for a 60 % improvement in IPC on divide-heavy code.

Frequency is limited by the memory architecture, not by logic depth. The critical path gets only half a clock period because the DTCM is negedge-clocked, as the single-cycle organisation of Figure 3 requires. The same logic would support roughly 50 MHz on a full period. Raising $f_{\max}$ meaningfully would therefore require pipelining — which is precisely what the bonus task asks for, and it is worth noting that the bottleneck we measured is the one that pipelining exists to remove.

Verification agreed across three independent paths. The simulated cycle count, the hardware cycle count and the RARS instruction count all reconcile on RV32IM/test1; the DTCM matches the golden model both in simulation and through ISMCE; and every peripheral behaviour observed in ModelSim was reproduced on the board, down to the input-capture value of 197 appearing identically in simulation, in a Signal-Tap trace, and in the DTCM readback.

The most instructive moment of the project was not a functional bug but a synthesis one: the Figure 10b synchronisers had been silently implemented in block RAM, which would have satisfied every functional test while quietly removing the metastability protection they exist to provide. Finding it required reading the Fitter's per-entity report rather than trusting the summary — and fixing it improved area, both clock domains, and correctness at once.

Full source, testbench, run scripts, memory images and Quartus outputs are in the accompanying submission archive; see [DOC/Readme.txt](#) for a description of every folder and file.